

Runbook: Production deploy failed (taskflow-api)

Trigger: Slack  from taskflow/main, or the stage **Deploy - Production** is red in Jenkins. **Goal:** users are served by a healthy color within **5 minutes**, then fix forward through the pipeline. **Owner:** on-call engineer (needs Jenkins admin and a PowerShell on the Jenkins host).

System facts

Thing	Value
Live Service	svc/taskflow in ns default, port 8080 → 3000. spec.selector.color decides who gets traffic
The two colors	deployment/taskflow-blue, deployment/taskflow-green (container name app) + per-color Services taskflow-blue, taskflow-green
Images	localhost:5000/taskflow-api:<sha7> (immutable, one tag per commit). Registry = container kind-registry
Health endpoint	GET /health/live → {"status":"success","data":{"status":"up"}}
Pipeline evidence	build artifacts svc-before.yaml (color before) and svc-after.yaml (color after)

Open PowerShell and define the helper first (every command below uses it):

```
function k { docker exec -i kind-control-plane kubectl --kubeconfig=/etc/kubernetes/admin.conf @args }
```

PowerShell gotchas: quote '--' in `k run ... '--' curl ...` (a bare -- is swallowed by PowerShell), and use `k set selector` instead of `k patch -p '{json}'` (PowerShell mangles the JSON quotes).

Step 0: Triage the failed build (1 min)

Open the build from the Slack link → **Console Output**, and search:

You find	Meaning	Go to
HEALTH_GATE_BLOCKED	Nothing was deployed. The pipeline is unhealthy (too many red builds)	Not an incident. Fix the failing builds. Stop.
Aborted at <i>Deploy to production?</i>	Nothing was deployed	Stop.
AUTOMATIC_ROLLBACK + Service now serving: <color>	The pipeline already rolled back	Step 2 (verify)
Unable to connect to the server, Could not find credentials entry, Jenkins restarted mid-deploy, or the log ends abruptly	Rollback may not have run	Step 1

Write down the two colors from the log line `Live = <PREV>`. Deploying `<image>` to `idle color <NEXT>`. **PREV = the good color, NEXT = the color that failed.**

Step 1: Is the cluster up? (1 min)

```
docker ps --filter name=kind --format "{{.Names}} {{.Status}}" # kind-control-plane + kind-registry must be Up
k get nodes # STATUS Ready
```

If not: `docker start kind-control-plane kind-registry`, wait until `k get nodes` says Ready, then continue.

Step 2: Verify what users get right now (1 min)

```
k get svc taskflow -o jsonpath='{.spec.selector.color}' # which color is live?
k get deploy taskflow-blue taskflow-green -o wide # READY + IMAGES per color
k run rb-check --rm -i --restart=Never --image=curlimages/curl:8.10.1 '--' curl -sf http://taskflow:8080/health/live
```

All three OK (live color = PREV, its READY is 1/1, and curl prints "status": "up"): users are fine. Go to **Step 4. Anything else:** Step 3.

Step 3: Manual rollback (only if Step 2 failed) (2 min)

3a. Pick the healthy color. Usually PREV. Confirm it is ready and on a known-good image:

```
k rollout status deployment/taskflow-<PREV> --timeout=10s
k get deploy taskflow-<PREV> -o jsonpath='{.spec.template.spec.containers[0].image}'
```

The image tag should equal the commit of the last green `main` build (Jenkins → last  build → `svc-after.yaml` shows its color).

3b. Send traffic to it.

```
k set selector svc taskflow "app=taskflow,color=<PREV>"
```

3c. Undo the half-deployed color, so the next deploy starts from a clean state:

```
k rollout undo deployment/taskflow-<NEXT>
```

3d. Only if BOTH colors are broken: redeploy a known-good tag. The registry keeps every tag.

```
curl.exe -s http://localhost:5000/v2/taskflow-api/tags/list
k set image deployment/taskflow-<PREV> app=localhost:5000/taskflow-api:<good-sha7>
k rollout status deployment/taskflow-<PREV> --timeout=120s
k set selector svc taskflow "app=taskflow,color=<PREV>"
```

3e. Repeat **Step 2** until all three checks pass.

Step 4: Stop the bleeding and communicate (1 min)

- Don't approve any production deploy until Step 6 is done. **Abort** pending *Deploy to production?* prompts.
- Postin #taskflow-ci: Prod deploy #N failed. Live = <color> on <sha7>, healthy. Investigating: <name>.

Step 5: Find the cause (users are safe now)

```
k describe pod -l color=<NEXT> # events: ImagePull, OOMKilled, probe failures
k logs deploy/taskflow-<NEXT> --previous # crash output (config.ts names missing env vars)
k get events --sort-by=.lastTimestamp | Select-Object -Last 20
```

Symptom	Likely cause	Fix
ErrImagePull localhost:5000/...	registry down, or the containerd mirror was lost	docker start kind-registry; re-copy k8s/hosts.toml into the node (Lab 07)
CrashLoopBackOff	app can't reach postgres/redis, or an env var is wrong	k get pods (deps Running?), check secret taskflow-env
Rollout timeout, pod Running but 0/1 Ready	/health/live failing	k logs deploy/taskflow-<NEXT>
Smoke test curl: (7) / (22)	app up but the endpoint is broken	app logs; compare with the last good commit
Jenkins: Unable to connect to the server	kind down, or the kind-kubeconfig credential is wrong	Step 1; re-create the credential from kind get kubeconfig --internal (Lab 07)

Step 6: Fix forward, through the pipeline

Fix on a branch → PR → develop (staging + E2E) → main. Or run `git revert <bad-commit>` on main. The full pipeline runs again (all gates, health gate, approval). **Never** leave a hand-made cluster change in place: the next deploy overwrites it, and nobody reviewed it.

Step 7: Close

- Step 2 passes. Post Resolved: <color> live on <sha7> with a link to the failed build.
- Within 2 days: a short postmortem (what happened, why, why no gate caught it, one action item).

Dry run (proves this runbook is executable)

- Jenkins → taskflow → main → **Build with Parameters** → BREAK_DEPLOY = true → approve.
- The idle color gets the broken image → rollout times out → console shows AUTOMATIC ROLLBACK / Service now serving: <PREV>.
- Follow Steps 0 → 2 → 4 and time yourself (target: under 5 min). To practice Step 3, run `k set selector svc taskflow "app=taskflow,color=<NEXT>"` yourself first (this breaks prod on purpose), then recover.
- Rebuild main without the parameter: the pipeline deploys a good image again.